



## **SIMATS ENGINEERING ASSESSMENT TOOL 3**

**COURSE CODE: CSA1407**

**COURSE NAME: Compiler Design**

---

### **COURSE OUTCOME COVERED**

***CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)***

*Assessment Tool Weightage: 10%*

---

**QUESTIONS:**

**Total Marks:50**

Annexure A

### **SCENARIO BASED TASK**

---

#### ***Code of Conduct:***

*I N.SANDIYANITTIANANDAM (192571029) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.  
I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

**N.SANDIYA**

## Annexure A

### SCENARIO-BASED TASK

#### QUESTIONS:

- 1.Scenario: Parsing Table Conflict in LL(1) While designing an LL(1) parser, a student constructs a parsing table and finds multiple entries in a single cell for a non-terminal and input symbol combination. This leads to confusion during parsing. Analyze how the syntax analyzer uses FIRST and FOLLOW sets in this process, identify the reason for the conflict, and explain how it affects deterministic parsing. Conclude by suggesting what decision should be taken to modify the grammar so that it becomes suitable for LL(1) parsing.
- 2.Scenario: Incorrect Parse Tree Generation A syntax analyzer generates a parse tree for the expression  $id + id - id$ , but the evaluation gives incorrect results due to improper grouping of operators. Analyze how the syntax analyzer constructs the parse tree, identify the issue related to associativity, and explain how grammar rules influence the structure of the tree. Finally, discuss what decision should be taken to enforce correct associativity and ensure accurate parsing.
- 3.Scenario: Invalid Expression Handling A syntax analyzer in a compiler receives the token sequence corresponding to the expression  $a + * b$  from the lexical analyzer. Although all tokens are valid individually, the parser fails to generate a parse tree and reports an error. In this situation, analyze how the syntax analyzer identifies the error based on grammar rules, determine the key issue in the token arrangement, and explain how parsing techniques such as predictive or shift-reduce parsing handle such cases. Further, discuss what decision should be taken to improve error detection and recovery while maintaining clarity in parsing.
4. Scenario: Ambiguity in Conditional Statements A syntax analyzer is designed using the grammar  $S \rightarrow \text{if } E \text{ then } S \mid \text{if } E \text{ then } S \text{ else } S \mid a$ . While constructing the parsing table, conflicts arise, and the parser cannot decide which production to apply for certain inputs. In this scenario, explain how the syntax analyzer encounters ambiguity, identify the key issue known as the dangling else problem, and discuss how it impacts parsing. Suggest an appropriate modification or parsing strategy to resolve the ambiguity and ensure correct association of else clauses.
- 5.Scenario: Operator Precedence Conflict A parser is designed for arithmetic expressions using the grammar  $E \rightarrow E + E \mid E * E \mid id$ . While parsing the input  $id + id * id$ , the syntax analyzer produces an incorrect parse tree due to ambiguity in operator precedence. Examine how the syntax analyzer processes this input, identify the issue caused by the grammar, and explain how precedence and associativity rules should be applied. Based on this, suggest what changes must be made to the grammar or parsing technique to ensure correct evaluation.

## RUBRICS FOR CALCULATION:

### Scenario based assessment

| Criteria                     | Weight age (%) | Level 4 (Exemplary)                                                       | Level 3 (Proficient)                               | Level 2 (Developing)                              | Level 1 (Beginning)                               |
|------------------------------|----------------|---------------------------------------------------------------------------|----------------------------------------------------|---------------------------------------------------|---------------------------------------------------|
| Understanding of Scenario    | 20%            | Demonstrates clear and complete understanding of the scenario and context | Shows good understanding with minor gaps           | Partial understanding; misses key aspects         | Misinterprets or fails to understand the scenario |
| Identification of Key Issues | 20%            | Accurately identifies all relevant issues and factors involved            | Identifies most key issues with minor omissions    | Identifies limited issues; some irrelevant points | Unable to identify key issues                     |
| Application of Concepts      | 25%            | Applies appropriate concepts effectively to address the scenario          | Applies concepts with minor errors                 | Limited or partially correct application          | Unable to apply relevant concepts                 |
| Decision Making              | 20%            | Makes well-reasoned, logical decisions with strong rationale              | Makes reasonable decisions with some justification | Makes weak decisions with limited justification   | No clear decisions made or rationale provided     |
| Clarity of Response          | 15%            | Response is clear, structured, and logically presented                    | Generally clear with minor issues                  | Somewhat unclear or poorly structured             | Disorganized and difficult to understand          |

## CO2-AT3 Assessment tool

N. Sandiyarittiamandan  
192511029  
CSA0402  
Compiler design

### Q) Sunario: Parsing table conflict in LL(1)

In an LL(1) parser, the parsing table is constructed using the FIRST and FOLLOW sets of the grammar. The FIRST set determines the terminals that can appear at the beginning of strings derived from a production, while the FOLLOW set contains terminals that can appear immediately after a non-terminal. During parsing table construction, each production is placed in the table based on these sets.

Since LL(1) parsing is deterministic, every table entry must contain only one production. Multiple entries make the parser non-deterministic and may lead to incorrect parsing or parser failure.

Decision: The grammar should be modified by eliminating left recursion, performing left factoring and ensuring that the FIRST and FOLLOW sets do not overlap.

2) Scenario: Incorrect parse tree Generation:

The syntax analyzer constructs a parse tree by applying grammar production according to the input tokens for the expression  $id + id - id$ , the parse tree determines the order in which operations are evaluated. If the grammar does not specify operator associativity, the expression may be grouped incorrectly. For example, it may be interpreted as  $id + (id - id)$  instead of the correct  $(id + id) - id$ . This results in incorrect evaluation because the operator is not associated properly.

Decision: The grammar should enforce left associativity for addition and subtraction. For  $y$ :

$$E \rightarrow E + T \mid E - T \mid T$$

$$T \rightarrow id.$$

This ensures expression are evaluated from left to right, producing the correct parse tree and accurate results.



Scenario: Invalid expression Handling:

The lexical Analyzer correctly identifies the tokens in the expression  $a + * b$  as identifiers and operators. However, the syntax analyzer checks whether these tokens follow the grammar rules.

The error occurs because the grammar expects an operand after the  $+$  operator, but another operator  $*$  appears instead. Therefore, the token sequence violates the grammar, making it impossible to construct a valid parse tree.

In predictive parsing (11), the parser detects error when no valid production exists in the parsing table for the current input symbol. In shift reduce parsing (12) the parser detects the error when no valid shift or reduce action is available.

Conclusion:

The parser should implement error detection and recovery techniques, such as panic mode recovery, phrase-structured recovery or synchronization using follow sets.



Engineer to Excel



# SAVEETHA

INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES  
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



4) Scenario: Ambiguity in conditional statement:

The grammar:

- $S \rightarrow \text{if } E \text{ then } S$
- $S \rightarrow \text{if } E \text{ then else } S$
- $S \rightarrow a$

is ambiguous because the parser cannot determine which if statement ~~an~~ else belongs to. This is known as the dangling else problem.

When constructing the parsing table, conflicts arise when both productions begin with the same prefix.

$\text{if } E \text{ then } S,$

The ambiguity affects deterministic parsing by introducing parsing conflicts and producing multiple possible parse trees.

Resolution: The grammar should be rewritten to distinguish matched and unmatched statements for example.

$S \rightarrow M/V$

$M \rightarrow \text{if } \text{ then } M \text{ else } M/a$

$V \rightarrow \text{if } E \text{ then } S \mid \text{if } E \text{ then } M \text{ else } V.$

Scenario: operator precedence conflict:

The grammar:

- $E \rightarrow E + E$
- $E \rightarrow E * E$
- $E \rightarrow id$

is ambiguous because it does not define operator precedence or associativity.

For the input  $id + id * id$ , two different parse trees are possible.

- $(id + id) * id$
- $id + (id * id)$

The correct evaluation requires multiplication to have higher precedence than addition. Since the grammar does not specify the rule, the parser cannot determine the correct parse tree.

Revision:

- $E \rightarrow E + T \mid T$
- $T \rightarrow T * F \mid F$
- $F \rightarrow id$

Thus:

- Multiplication has higher precedence than addition.
- Addition and multiplication are left associative.